Path Traversal — Interview Prep Notes

1. Describe Path Traversal logically

- Untrusted input used to construct filesystem path
- App trusts input stays within intended dir; OS doesn't enforce that
- `../` = parent dir at OS level, not app level
- Concatenation happens before resolution → validation on wrong representation
- Root: validate string ≠ validate resolved path

2. OWASP / CWE

- **CWE-22**: Improper Limitation of a Pathname to a Restricted Directory
- Related: CWE-23 (Relative Path Traversal), CWE-36 (Absolute Path Traversal), CWE-434 (Unrestricted Upload — if chained), CWE-59 (symlink)
- **OWASP Top 10 2021**: A01 – Broken Access Control
- **OWASP API Top 10**: API1 (BOLA) adjacent if resource ID = filename

3. Common variants

- Basic `../../../etc/passwd`
- URL encoded `..%2f`
- Double encoded `..%252f`
- Unicode/overlong UTF-8 `..%c0%af`
- Backslash (Windows) `..\`
- Non-recursive filter bypass `....//`
- Null byte `file.php%00.jpg` (legacy, filter bypass not traversal itself)
- Absolute path override `/etc/passwd`
- Zip Slip (archive entries)
- Symlink-based escape
- Second-order (stored filename used later)

4. Attack surface / entry points

- File download/view/export endpoints (`?file=`, `?path=`, `?template=`, `?lang=`)
- Upload endpoints (filename/destination control)
- Archive extraction (zip/tar install, backup restore)
- Template engines / include mechanisms
- Log viewers, config loaders, module loaders
- Headers/cookies (rare)
- Second-order sinks (stored value used by background job)
- Non-HTTP: CLI args, gRPC/GraphQL fields, FTP

5. Testing workflow (condensed)

Phase	Goal	Manual	Burp	ZAP	Nuclei
Recon	find candidate params	grep site map	Site Map/Content Discovery	Spider/AJAX Spider	`-tags file,exposure`
Baseline	normal behavior	valid vs invalid value	Repeater + Comparer	Compare view	debug-req-resp
FS touch confirm	reaches file I/O	slash/case mutation	Repeater/Intruder	Fuzzer	custom matcher for error strings
Raw probe	zero filtering check	`../../../etc/passwd`	Repeater, grep match	Active Scan (Path Traversal rule only)	`file/lfi/` templates
Filter characterization	type of filter	single vs nested vs absolute path tests	Intruder Sniper + Comparer	Fuzzer custom list	custom multi-matcher template
Encoding escalation	bypass filter	URL→double→unicode→backslash, one at a time	Intruder payload list, sort by length	built-in traversal payload category	`-fuzz` flag, community lfi-fuzzing templates
Alt vectors	if canonicalization proper	Zip Slip / symlink craft	manual multipart craft	Script Console chaining	Zip Slip tag / workflow YAML
Write impact	escalate severity	upload w/ traversal filename → verify via GET	Repeater/Intruder chain	manual + script	2-step workflow template

6. Evidence to collect

- Raw request + raw response (file content, e.g. `root:x:0:0`)
- Baseline (benign input) vs exploited response diff
- Target-specific file content (not generic) — proves real read
- Depth-variation proof (`../` count changes outcome)
- Exact encoding/bypass technique used
- Impact escalation proof (RCE cmd output / write location confirmed)
- Auth context if cross-tenant/privilege boundary crossed
- curl/Repeater-exportable repro + timestamp

7. Good PoC

- Payload + full request/response

- Baseline comparison screenshot
- File content proof (redacted/minimal, not full dump)
- If RCE: command execution output (`id` / `whoami`)
- If write: follow-up GET proving file landed outside intended dir
- Step-by-step reproducible curl commands

8. Attack chains

- LFI + log poisoning (User-Agent → access.log → include) → RCE
- LFI + PHP session file inclusion → RCE
- LFI + PHP wrappers (`php://filter`, `data://`, `expect://`)
- Arbitrary write → webshell drop → RCE
- Zip Slip → cron/SSH key overwrite → persistence/RCE
- Config/secret read → cloud/infra credential pivot
- Session/auth file read → account takeover

9. False positives

- `../` in legit non-path context (regex, code snippets, comments)
- Encoded chars in unrelated tokens (JWT/base64) matching pattern
- Generic error pages containing file-signature-like strings
- Legit octet-stream/binary Content-Type on benign endpoints
- Own/authorized scanner traffic (Nuclei/ZAP/Burp) not excluded
- Framework internal realpath() calls in logs (never reach unsafe sink)
- Package manager / build tool paths in server logs

10. Prerequisites

- Unsanitized input reaching filesystem call
- Filter absent, naive, or pre-canonicalization
- Target file exists and is readable/writable by service account
- Service account permissions bound impact ceiling
- For RCE: additional sink (include/log/session/writable+executable path)
- Network reachability / auth requirement level
- No compensating control (WAF, chroot, SELinux, RO mount) blocking

11. What attacker gains

- Info disclosure: OS files, source code, config/secrets, logs
- Credential harvesting → lateral movement
- Auth/session bypass
- RCE (via LFI chains or arbitrary write)
- DoS (integrity corruption if write-capable)

12. CIA impact table

Variant	C	I	A	Example Vector
Read-only, non-sensitive file	L	N	N	`C:L/I:N/A:N`
Read source code/config	H	N	N	`C:H/I:N/A:N`
Read secrets/creds	H	N	N (Scope Changed)	`C:H/I:N/A:N`, S:C
Arbitrary write (no RCE proven)	N	H	L	`C:N/I:H/A:L`
LFI/write chained to RCE	H	H	H	`C:H/I:H/A:H`

13. Typical CVSS

- Unauth read secrets: **9.4** (Critical) `AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:N/A:N`
- Unauth read source: **7.5** (High) `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`
- Arbitrary write: **8.1** (High) `AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:L`
- LFI→RCE: **10.0** (Critical) `AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H`
- Cross-tenant read (auth req): **8.7** (High), S:C

14. Business impact (financial app context)

- Config/secret read → DB/core banking credential exposure → lateral compromise
- PCI-DSS scope: cardholder data file exposure = major violation, fines
- Cross-tenant file read = regulatory breach notification (RBI/GDPR-equivalent)
- RCE on financial host = transaction integrity risk, fraud potential
- Reputational + contractual damage disproportionate to CVSS in regulated sector
- Incident response, forensics, and mandatory disclosure costs

15. Detection signals (IOCs)

- `../`, encoded/double-encoded variants in logs
- 200 OK on normally-4xx endpoint
- Response containing `root:x:0:0`, `[boot loader]`
- Content-Type/size anomaly vs baseline
- Sequential depth-incrementing requests (scanner pattern)
- FS-level: web process opening files outside webroot (auditd/Sysmon)
- WAF rule hits (CRS 930100 series)
- Zip Slip: extracted path resolves outside extraction root

16. Root cause (code logic)

```
path = BASE_DIR + "/" + user_input   // raw concat
  file = OS.open(path)                  // resolved AFTER concat, no check
```

- Validation (if any) on raw string; filesystem resolves separately
- Naive blacklist (`../` strip) fails: `....//`, encoding, absolute path, symlink

17. Remediation / secure coding

```
resolved = realpath(join(BASE_DIR, user_input))
  base     = realpath(BASE_DIR)
  if not resolved.startswith(base + sep): reject
```

- Canonicalize BEFORE boundary check, not after
- Prefer indirect reference (ID → DB lookup path) over raw filename
- Allowlist filenames/extensions, never blacklist
- Use framework-safe static file servers, not hand-rolled open()
- Validate archive entries per-entry, same canonicalize+check logic
- Disable script execution in upload dirs (server config)

18. Compensating controls

- WAF (ModSecurity CRS, Cloudflare)
- Chroot/containerization, minimal filesystem
- SELinux/AppArmor MAC
- Read-only mounts for app/webroot
- Least-privilege service account, no shell
- FIM for post-exploit detection
- RASP at runtime

19. Recurring issue = SDLC signal

- Indicates missing secure file-handling pattern/library org-wide
- No SAST gate for taint→file-sink flows
- Devs default to raw concatenation, no canonicalization standard
- Same systemic pattern as IDOR: missing "never trust client-supplied resource reference" principle in SDLC
- Signals need for shared safe-access library + mandatory code review checklist item

20. Upstream improvements

- Semgrep taint-mode rule: source (request params) → sink (file I/O) → sanitizer (realpath/normalize)
- CodeQL dataflow query (`java/path-injection` or custom)
- Shared internal `safe_file_access` library per language
- Staged rollout: WARN → blocking on diff → full blocking
- Linter/pre-commit check for raw `open()` / `fopen()` without sanitizer wrapper
- Framework-level static file server usage enforced via design checklist
- Dev education: inline PR message linking shared library

21. Sample triage response

> *"Confirmed CWE-22 Path Traversal on `[endpoint]`. Payload `[encoded payload]` bypasses [filter type], resulting in disclosure of `[target file]` (see attached request/response). Baseline comparison confirms this deviates from expected 403 behavior. Impact: [read/write/RCE], CVSS `[vector]` (`[score]`, `[rating]`). Recommend immediate remediation per attached guidance; retest requested post-fix."*

22. Sample developer remediation note

> *"Root cause: `[param]` concatenated directly into file path without canonicalization (`[file:line]`). Fix: resolve full path via `realpath()` / `getCanonicalPath()` / `path.resolve()`, then verify result is descendant of intended base dir before file I/O — see `safe_file_access` shared lib for reference implementation. Do not rely on blacklist filtering of `../`. Retest with encoded payloads (`..%2f`, `..%252f`, `....//`) post-fix."*

23. Real-world example

- **CVE-2021-41773 / CVE-2021-42013** — Apache HTTP Server 2.4.49/2.4.50: path traversal via `%2e%2e/` (double-encoding bypass of normalization) in cgi-scripts, chained to RCE — widely exploited in the wild within days of disclosure
- Also reference: **CVE-2019-19781** (Citrix ADC/Gateway) — path traversal → RCE, mass exploited pre-patch